

O QUE FOI FEITO

Fase 3 — Consolidação, Testes e Funcionalidades Avançadas

Artificial Intelligence in a Learning Organization

Projeto de Engenharia Informática — Universidade Aberta

Ricardo & Carla

Baseado no framework de Santos & Mamede (2026)

Maio 2026

Índice

1. Enquadramento e Objetivos da Fase 3
2. Evolução face à Fase 2
3. Testes Automatizados (pytest)
4. Exportação de Relatórios PDF (ReportLab)
5. Painel de Administração
6. Comparação Temporal de Avaliações
7. Motor de Recomendação de Ferramentas IA
8. Perfil de Utilizador
9. Melhorias de Robustez e Arquitetura
10. Estrutura de Ficheiros
11. Instruções de Instalação e Execução
12. Instruções para Deployment Real
13. Resultados dos Testes
14. Trabalho Futuro

1. Enquadramento e Objetivos da Fase 3

A Fase 3 do projeto AILO representa a consolidação da plataforma, evoluindo o MVP funcional da Fase 2 com testes automatizados, funcionalidades avançadas e melhorias de robustez. O projeto é desenvolvido por Ricardo e Carla no âmbito do curso de Engenharia Informática da Universidade Aberta, baseado no framework teórico de Santos & Mamede (2026).

Objetivos principais:

- Implementar testes automatizados com pytest (37 testes)
- Exportação de relatórios de diagnóstico em PDF profissional (ReportLab)
- Painel de administração para gestão de indicadores, ferramentas e estatísticas
- Comparação temporal entre avaliações da mesma organização
- Motor de recomendação de ferramentas IA baseado nos resultados
- Gestão de perfil de utilizador
- Melhorias de robustez: error handlers, validação, paginação

2. Evolução face à Fase 2

A Fase 3 partiu da base de código da Fase 2 e adicionou as seguintes evoluções:

Área	Fase 2	Fase 3
Testes	Validação manual	37 testes automatizados (pytest)
Relatórios PDF	Apenas HTML	PDF profissional com ReportLab
Administração	Sem painel admin	Dashboard com estatísticas e CRUD
Comparação	Avaliação única	Sobreposição temporal multi-avaliação
Recomendações	Sem recomendações	Motor baseado em scores por camada
Perfil	Sem gestão de perfil	Edição de nome e alteração de password
Error handling	Básico	Handlers JSON para 404/500 + validação
Config	Dev/Prod	Dev/Prod/Testing (in-memory SQLite)
Rotas backend	9 blueprints	11 blueprints (+admin, +perfil)
Páginas frontend	7 páginas	10 páginas (+admin, +comparação, +perfil)

3. Testes Automatizados (pytest)

Foram implementados 37 testes automatizados distribuídos por 6 ficheiros, utilizando pytest com fixtures partilhados e base de dados in-memory (SQLite):

Ficheiro	Testes	Cobertura
confest.py	—	Fixtures: app, client, db_session, auth_header, org_id, avaliacao_id
test_auth.py	11	Registo (sucesso, email duplicado, campos em falta, email inválido, password fraca), Login (sucesso, credenciais inválidas)
test_organizacoes.py	9	CRUD completo: criar (sucesso, sem campos, sem auth), listar (vazio, com dados), editar (sucesso, inexistente)
test_avaliacoes.py	8	Criar (sucesso, sem org, sem auth), listar, detalhar, respostas (individual, batch, score inválido)
test_scoring.py	7	Classificação de níveis (5 níveis), scoring com avaliação inexistente, scoring com respostas completas
test_interdependencias.py	2	Análise completa com dados reais (fluxo API), função direta com avaliação inexistente

Configuração de testes: A classe `TestingConfig` utiliza SQLite in-memory (`sqlite:///memory:`) para isolamento total. Os fixtures com scope `'session'` populam a BD uma vez com os 51 indicadores AILO; fixtures com scope `'function'` garantem independência entre testes.

4. Exportação de Relatórios PDF (ReportLab)

O serviço `services/pdf_report.py` gera relatórios PDF profissionais com a biblioteca ReportLab. Funcionalidades:

- **Capa personalizada** com nome da organização, data e score global
- **Resumo executivo** com scores por camada em tabela formatada
- **Detalhes por camada:** score, nível, pontos fortes, lacunas e recomendações
- **Interdependências:** tabela com pares críticos, tipo de relação e impacto
- **Endpoint REST:** GET `/api/v1/avaliacoes/:id/relatorio/pdf` — retorna o ficheiro PDF como download
- **Botão na UI:** Botão '■ Download PDF' na página de resultados

5. Painel de Administração

O painel de administração (`routes/admin.py` + `pages/admin.html`) fornece:

- **Dashboard de estatísticas:** Total de utilizadores, organizações, avaliações e score médio global
- **Gestão de indicadores:** Edição de pesos dos indicadores por camada
- **Gestão de ferramentas IA:** CRUD completo do catálogo de ferramentas (nome, descrição, área, custo, complexidade, URL)
- **Lista de utilizadores:** Visualização de todos os utilizadores registados
- **Proteção:** Todas as rotas admin requerem papel 'admin' via decorador `@admin_required`

6. Comparação Temporal de Avaliações

A funcionalidade de comparação temporal ([pages/comparacao.html](#)) permite sobrepor múltiplas avaliações da mesma organização para visualizar a evolução da maturidade ao longo do tempo.

Backend: Endpoint GET `/api/v1/organizacoes/:id/comparacao` que retorna todas as avaliações concluídas com os respectivos resultados por camada.

Frontend: Dois gráficos Chart.js:

- **Radar multi-dataset:** Sobreposição dos perfis de cada avaliação (uma cor por avaliação)
- **Gráfico de linhas:** Evolução do score global ao longo das avaliações
- **Tabela comparativa:** Scores lado a lado com setas de tendência ($\uparrow \downarrow \rightarrow$)

7. Motor de Recomendação de Ferramentas IA

O serviço `services/recomendacoes.py` cruza os resultados do diagnóstico com o catálogo de ferramentas IA para gerar recomendações personalizadas.

Algoritmo:

- Para cada camada com score < 3.5 (abaixo de 'Gerido'), identifica ferramentas IA relevantes
- Calcula prioridade com base no gap $(5.0 - \text{score}) \times \text{peso da camada}$
- Ordena recomendações por prioridade decrescente
- Gera justificação contextualizada para cada recomendação

Endpoint: GET `/api/v1/avaliacoes/:id/recomendacoes`

UI: Secção dedicada na página de resultados com cards de ferramentas recomendadas, organizadas por prioridade.

8. Perfil de Utilizador

A página de perfil (routes/perfil.py + pages/perfil.html) permite:

- **Ver dados pessoais:** Nome, email e papel
- **Editar nome:** PUT /api/v1/perfil com validação
- **Alterar password:** PUT /api/v1/perfil/password com verificação da password atual

9. Melhorias de Robustez e Arquitetura

- **Error handlers JSON:** Respostas 404 e 500 em formato JSON em vez de HTML
- **TestingConfig:** Configuração dedicada para testes com BD in-memory
- **Geração lazy de recomendações:** Se as recomendações não existem ao consultar, são geradas automaticamente
- **Navbar dinâmica:** Links para admin (visível apenas para administradores), perfil e comparação
- **Integração no finalizar:** Ao finalizar uma avaliação, o sistema executa automaticamente scoring → interdependências → recomendações → relatório

10. Estrutura de Ficheiros

- Fase_3/backend/app/__init__.py — App factory (11 blueprints, error handlers)
- Fase_3/backend/app/config.py — Configurações dev/prod/testing
- Fase_3/backend/app/models/ — 8 modelos SQLAlchemy (12+ tabelas)
- Fase_3/backend/app/routes/ — 11 ficheiros de rotas (auth, org, ailo, avaliações, respostas, resultados, chat, relatórios, admin, perfil)
- Fase_3/backend/app/services/ — 6 serviços (scoring, interdependências, IA, relatórios, PDF, recomendações)
- Fase_3/backend/app/utils/ — Auth JWT, decoradores, validadores
- Fase_3/backend/tests/ — 6 ficheiros de testes (37 testes pytest)
- Fase_3/backend/seeds/ — Dados iniciais do framework AILO
- Fase_3/frontend/pages/ — 10 páginas HTML (+ admin, comparação, perfil)
- Fase_3/frontend/css/ — Design system premium dark theme
- Fase_3/frontend/js/ — API wrapper e app module

11. Instruções de Instalação e Execução

11.1 Pré-requisitos

- Python 3.10+ instalado

11.2 Instalação e Execução

```
$ cd Fase_3/backend  
$ python3 -m venv venv  
$ source venv/bin/activate # Linux/Mac  
$ pip install -r requirements.txt  
$ cp .env.example .env # Editar com as suas configurações  
$ python seed.py # Popular a base de dados  
$ python run.py # Iniciar → http://localhost:5000
```

11.3 Executar Testes

```
$ python -m pytest tests/ -v
```

Resultado esperado: 37 passed

12. Instruções para Deployment Real

- **Base de Dados:** Migrar para PostgreSQL (alterar DATABASE_URL no .env)
- **Servidor WSGI:** gunicorn -w 4 -b 0.0.0.0:5000 'app:create_app("production")'
- **Proxy Reverso:** Nginx com SSL (Let's Encrypt)
- **SECRET_KEY:** Gerar chave forte (python -c "import secrets; print(secrets.token_hex(32))")
- **GEMINI_API_KEY:** Obter em <https://aistudio.google.com/apikey>
- **Criar utilizador admin:** Inserir manualmente na BD um utilizador com papel='admin'

13. Resultados dos Testes

Todos os 37 testes passam com sucesso:

- test_auth.py: 11 passed ■
- test_organizacoes.py: 9 passed ■
- test_avaliacoes.py: 8 passed ■
- test_scoring.py: 7 passed ■
- test_interdependencias.py: 2 passed ■
- Servidor Flask inicia sem erros ■

14. Trabalho Futuro

- Integração com LMS (Moodle) via xAPI/LTI
- Benchmarking setorial (comparação entre organizações do mesmo setor)
- Aplicação mobile (PWA — Progressive Web App)
- Sistema de notificações por email
- Migração de base de dados com Alembic
- CI/CD pipeline com GitHub Actions
- Docker + docker-compose para deployment containerizado
- Expansão do catálogo de ferramentas IA com filtragem inteligente
- Dashboard analytics com métricas de utilização da plataforma

—— Fim do documento ——

Gerado automaticamente pela plataforma AILO · Maio 2026